

Array Rotation Programs - Step by Step Process

Program 1: Left Rotation by 1 Position

Code:

```
def left(arr):  
    n = len(arr)  
    temp = arr[0]          # store first element  
    for i in range(1, n):  
        arr[i-1] = arr[i]  # shift left  
    arr[n-1] = temp        # put first element at the end  
    return arr  
  
arr = [1, 2, 3, 4, 5]  
print(left(arr))
```

Step-by-Step Execution:

Initial State:

- arr = [1, 2, 3, 4, 5]
- n = 5

Step 1: Store first element

- temp = arr[0] = 1
- Array remains: [1, 2, 3, 4, 5]

Step 2: Shift elements left using loop

- i = 1: arr[0] = arr[1] → arr = [2, 2, 3, 4, 5]
- i = 2: arr[1] = arr[2] → arr = [2, 3, 3, 4, 5]
- i = 3: arr[2] = arr[3] → arr = [2, 3, 4, 4, 5]
- i = 4: arr[3] = arr[4] → arr = [2, 3, 4, 5, 5]

Step 3: Place first element at the end

- arr[4] = temp = 1 → arr = [2, 3, 4, 5, 1]

Output: [2, 3, 4, 5, 1]

Program 2: Right Rotation by 1 Position

Code:

```
def right(arr):
    n = len(arr)
    temp = arr[n-1]          # store last element
    for i in range(n-1, 0, -1): # move elements to the right
        arr[i] = arr[i-1]
    arr[0] = temp            # put last element at the beginning
    return arr

arr = [1, 2, 3, 4, 5]
print(right(arr))
```

Step-by-Step Execution:

Initial State:

- arr = [1, 2, 3, 4, 5]
- n = 5

Step 1: Store last element

- temp = arr[4] = 5
- Array remains: [1, 2, 3, 4, 5]

Step 2: Shift elements right using loop

- i = 4: arr[4] = arr[3] → arr = [1, 2, 3, 4, 4]
- i = 3: arr[3] = arr[2] → arr = [1, 2, 3, 3, 4]
- i = 2: arr[2] = arr[1] → arr = [1, 2, 2, 3, 4]
- i = 1: arr[1] = arr[0] → arr = [1, 1, 2, 3, 4]

Step 3: Place last element at the beginning

- arr[0] = temp = 5 → arr = [5, 1, 2, 3, 4]

Output: [5, 1, 2, 3, 4]

Program 3: Left Rotation by d Positions

Code:

```
def left(arr,n,d):
    temp = arr[:d]
    for i in range(d,n):
        arr[i-d] = arr[i]    # 4 5 6 7
    for i in range(n-d,n):
        arr[i] = temp[i-(n-d)] #1 2 3
    return arr

arr = [1,2,3,4,5,6,7]
```

```
d = 3
n = len(arr)
print(left(arr,n,d))
```

Step-by-Step Execution:

Initial State:

- `arr = [1, 2, 3, 4, 5, 6, 7]`
- `n = 7, d = 3`

Step 1: Store first d elements

- `temp = arr[:3] = [1, 2, 3]`
- Array remains: `[1, 2, 3, 4, 5, 6, 7]`

Step 2: Shift remaining elements left by d positions

- `i = 3: arr[0] = arr[3] = 4 → arr = [4, 2, 3, 4, 5, 6, 7]`
- `i = 4: arr[1] = arr[4] = 5 → arr = [4, 5, 3, 4, 5, 6, 7]`
- `i = 5: arr[2] = arr[5] = 6 → arr = [4, 5, 6, 4, 5, 6, 7]`
- `i = 6: arr[3] = arr[6] = 7 → arr = [4, 5, 6, 7, 5, 6, 7]`

Step 3: Place stored elements at the end

- `i = 4: arr[4] = temp[0] = 1 → arr = [4, 5, 6, 7, 1, 6, 7]`
- `i = 5: arr[5] = temp[1] = 2 → arr = [4, 5, 6, 7, 1, 2, 7]`
- `i = 6: arr[6] = temp[2] = 3 → arr = [4, 5, 6, 7, 1, 2, 3]`

Output: `[4, 5, 6, 7, 1, 2, 3]`

Program 4: Right Rotation by d Positions

Code:

```
def right(arr,n,d):
    temp=arr[n-d:] # 5 6 7
    for i in range(n-d-1,-1,-1):
        arr[i+d]=arr[i] # 1 2 3 4
    for i in range(d):
        arr[i]=temp[i]
    return arr

arr = [1,2,3,4,5,6,7]
d = 3
n = len(arr)
print(right(arr,n,d)) #output: 5 6 7 1 2 3 4
```

Step-by-Step Execution:

Initial State:

- $arr = [1, 2, 3, 4, 5, 6, 7]$
- $n = 7, d = 3$

Step 1: Store last d elements

- $temp = arr[4:] = [5, 6, 7]$
- Array remains: $[1, 2, 3, 4, 5, 6, 7]$

Step 2: Shift first $(n-d)$ elements right by d positions

- $i = 3: arr[6] = arr[3] = 4 \rightarrow arr = [1, 2, 3, 4, 5, 6, 4]$
- $i = 2: arr[5] = arr[2] = 3 \rightarrow arr = [1, 2, 3, 4, 5, 3, 4]$
- $i = 1: arr[4] = arr[1] = 2 \rightarrow arr = [1, 2, 3, 4, 2, 3, 4]$
- $i = 0: arr[3] = arr[0] = 1 \rightarrow arr = [1, 2, 3, 1, 2, 3, 4]$

Step 3: Place stored elements at the beginning

- $i = 0: arr[0] = temp[0] = 5 \rightarrow arr = [5, 2, 3, 1, 2, 3, 4]$
- $i = 1: arr[1] = temp[1] = 6 \rightarrow arr = [5, 6, 3, 1, 2, 3, 4]$
- $i = 2: arr[2] = temp[2] = 7 \rightarrow arr = [5, 6, 7, 1, 2, 3, 4]$

Output: $[5, 6, 7, 1, 2, 3, 4]$

Bubble Sort Algorithm

Introduction

Bubble Sort is a simple comparison-based sorting algorithm. It works by repeatedly stepping through the list to be sorted, compares adjacent elements, and swaps them if they are in the wrong order. This process is repeated until the list is sorted.

Key Characteristics

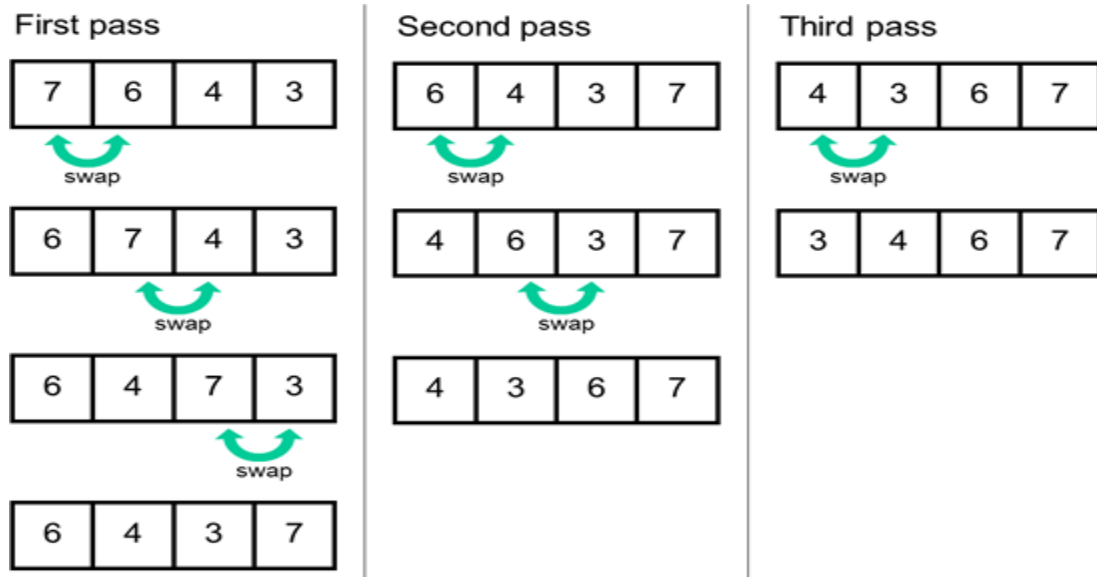
1. In-Place Sorting:

- Bubble Sort does not require additional memory and sorts the array in place.

2. Stable Sorting:
 - Bubble Sort preserves the relative order of equal elements.
3. Time Complexity:
 - **Best Case:** $O(n)$ (when the array is already sorted).
 - **Worst Case:** $O(n^2)$ (when the array is sorted in reverse order).
4. Space Complexity:
 - **$O(1)$:** Bubble Sort uses a constant amount of extra space.

How Bubble Sort Works:

1. **Start with the first element** and compare it with the next element.
2. If the first element is greater than the second, **swap them**.
3. Continue this comparison and swap through the array.
4. After each pass, the largest element is placed at the end.
5. Repeat this process for the remaining unsorted elements.



```
def sort(nums):  
    swapcount = 0  
    for i in range(len(nums)-1, 0, -1): # outer loop  
        for j in range(i):             # inner loop
```

```
    if nums[j] > nums[j+1]:    # check if swap needed
        temp = nums[j]
        nums[j] = nums[j+1]
        nums[j+1] = temp
        swapcount += 1
return swapcount    # return the count
```

```
nums = [5,4,3,2,1]
count = sort(nums)
print("Sorted list:", nums)
print("Total swaps:", count)
```

output:

Sorted list: [1, 2, 3, 4, 5]

Total swaps: 10

Selection Sort Algorithm

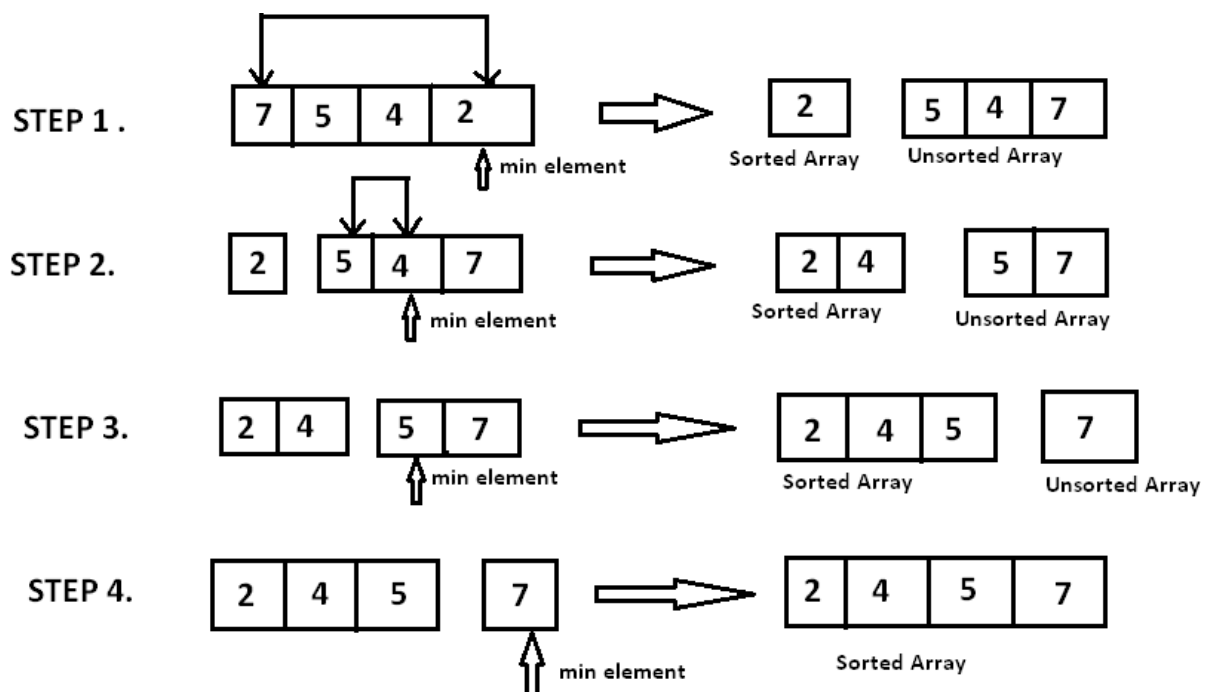
Introduction

Sorting is a fundamental problem in computer science where the goal is to arrange a set of data in a specific order. The **Selection Sort algorithm is one of the simplest sorting techniques used in sorting algorithms**. It works by repeatedly selecting the smallest (or largest, depending on the order) element from the unsorted portion of the list and swapping it with the first unsorted

element. Though simple, Selection Sort is not the most efficient for large datasets, but it offers a good understanding of sorting principles.

Selection Sort Algorithm Explanation

1. **Initialization:** Start with the first element of the array.
2. **Find the Minimum:** Search the unsorted portion of the array to find the smallest element.
3. **Swap:** Swap the found smallest element with the first unsorted element.
4. **Repeat:** Move to the next unsorted element and repeat the process until the entire array is sorted.



#Selection Sort Program

```
def sort(nums):  
    swapcount = 0  
    for i in range(len(nums)):  
        minpos = i  
        for j in range(i, len(nums)):  
            if nums[j] < nums[minpos]:
```

```

        minpos = j
    if minpos != i: # swap only if needed
        temp = nums[i]
        nums[i] = nums[minpos]
        nums[minpos] = temp
        swapcount += 1
    return swapcount

```

```

nums = [5, 4, 3, 2, 1]
count = sort(nums)
print("Sorted list:", nums)
print("Total swaps:", count)

```

output:

```

Sorted list: [1, 2, 3, 4, 5]
Total swaps: 2

```

Time Complexity

- **Best Case:** $O(n^2)$ (Even if the array is already sorted, the algorithm still performs comparisons).
- **Worst Case:** $O(n^2)$ (The array is sorted in reverse order).

Space Complexity

O(1): The algorithm uses a constant amount of extra space for variables like temp and Swapcount.

Insertion Sort Algorithm

Introduction

Sorting algorithms are essential tools in computer science for organizing data. One of the simplest and most intuitive sorting algorithms is **Insertion Sort**. It works by building a sorted array (or list) one element at a time, much like sorting playing cards in your hands. Despite its simplicity, Insertion Sort can be very efficient for small or nearly sorted datasets.

Insertion Sort Explanation

Insertion Sort works by iterating through the array and comparing the current element with the elements in the sorted portion of the array (on the left). It then inserts the current element into its correct position in the sorted portion.

Here's how it works:

1. Start with the second element (because a single element is already sorted).
2. Compare it with the previous elements. If it is smaller, shift those elements one position to the right.
3. Insert the element in its correct position.

```
def insertion_sort(arr):  
    swap_count = 0  
    n = len(arr)  
  
    for i in range(1, n):  
        key = arr[i]  
        j = i - 1  
  
        # Shift elements greater than key to the right  
        while j >= 0 and arr[j] > key:  
            arr[j + 1] = arr[j]  
            j -= 1  
            swap_count += 1 # count each shift  
  
        arr[j + 1] = key # insert key at correct position  
  
    return swap_count  
  
# Input  
arr = [5, 4, 3, 2, 1]
```

```
count = insertion_sort(arr)

print("Sorted array:", arr)

print("Number of swaps:", count)
```

output:

Sorted array: [1, 2, 3, 4, 5]
Number of swaps: 10

Time and Space Complexity

- Time Complexity:
 - **Best Case:** $O(n)$ (when the array is already sorted).
 - **Worst Case:** $O(n^2)$ (when the array is sorted in reverse order).
- Space Complexity:
 - **$O(1)$:** The algorithm uses a constant amount of extra space regardless of the input size.